

Prompt History — lab02_estiamba.py

Step 1 — Data setup

YOU

Load 288 fifteen-minute reservoir depth readings starting 2026-07-21. Build a numpy array, an hours-elapsed time axis, and matching timestamps. Assert the count is 288.

CLAUDE

Here's the data setup:

```
START_TS = datetime(2026, 7, 21, 0, 0)
DT_HOURS = 0.25

DEPTH = [14.18, 14.21, 14.22, 14.21, 14.21, ...] # 288 readings

h = np.array(DEPTH, dtype=float)
n = len(h)
t = np.arange(n) * DT_HOURS
timestamps = [START_TS + timedelta(hours=DT_HOURS * i) for i in range(n)]

assert n == 288, f"expected 288 readings, got {n}"
```

Step 2 — Finite-difference derivatives

YOU

Compute dh/dt and d^2h/dt^2 with finite differences (central inside, forward/backward at the ends). Find max dh/dt . Compare d^2 noise during $t=25-40h$ vs. the rest of the record.

CLAUDE

Here's the derivative section:

```
d1 = np.empty(n)
d1[0] = (h[1] - h[0]) / DT_HOURS
d1[-1] = (h[-1] - h[-2]) / DT_HOURS
d1[1:-1] = (h[2:] - h[:-2]) / (2 * DT_HOURS)

d2 = np.empty(n)
d2[0] = (h[2] - 2*h[1] + h[0]) / DT_HOURS**2
d2[-1] = (h[-1] - 2*h[-2] + h[-3]) / DT_HOURS**2
d2[1:-1] = (h[2:] - 2*h[1:-1] + h[:-2]) / DT_HOURS**2

i_max_rate = int(np.argmax(d1))
t_max_rate, v_max_rate = t[i_max_rate], d1[i_max_rate]

event = (t >= 25) & (t <= 40)
print(np.std(d2[event]), np.std(d2[-event])) # noise in vs. out of the event
```

Step 3 — Model and fit

YOU

Fit a sum of two logistic curves (rise, then drawdown) to the depth series with `curve_fit`, Levenberg-Marquardt, and a hand-picked initial guess.

CLAUDE

Here's the model and the fit:

```
def sigmoid(tt, a, k, t0):
    return a / (1 + np.exp(-k * (tt - t0)))

def model(tt, c, a1, k1, t01, a2, k2, t02):
    return c + sigmoid(tt, a1, k1, t01) + sigmoid(tt, a2, k2, t02)

p0 = [14.18, 7.0, 0.5, 32.0, -1.6, 0.15, 42.0]
popt, pcov = curve_fit(model, t, h, p0=p0, method="lm", maxfev=20000)

h_hat = model(t, *popt)
resid = h - h_hat
```

Step 4 — Fit statistics and residuals

YOU

Compute R2, SSE, SST, and per-parameter standard errors/t-stats/p-values. Check residuals for long same-sign runs and flag the largest one.

CLAUDE

Here's the statistics and residual-diagnostics section:

```
sse = np.sum(resid**2)
sst = np.sum((h - h.mean())**2)
r2 = 1 - sse / sst
dof = n - len(popt)
s_est = np.sqrt(sse / dof)

se = np.sqrt(np.diag(pcov))
tj = popt / se
pj = 2 * (1 - stats.t.cdf(np.abs(tj), dof))

signs = np.sign(resid)
signs[signs == 0] = 1
# ...walk `signs` and record each run of consecutive equal values...
longest_run = max(runs, key=lambda r: r[3])

max_abs_resid = np.max(np.abs(resid))
resid_vs_resolution = max_abs_resid / 0.01 # sensor resolution, metres
```

Step 5 — Area under the curve

YOU

Integrate the fitted curve with quad and the raw points with the trapezoid rule, compare them, and note the result is metre-hours, not volume.

CLAUDE

Here's the integration section:

```
area_quad, area_quad_err = quad(lambda tt: model(tt, *popt), t[0], t[-1])
area_trap = np.trapezoid(h, t)
area_gap = area_quad - area_trap

# quad integrates the smooth fitted curve; trapezoid connects the raw,
# rounded points with straight lines - they diverge most on the sharp
# fill/release flank. Units are m*h (a time-integral of stage), not volume.
```

Step 6 — Charts

YOU

Plot the raw series, derivatives, fit vs. data, residuals, and area under the curve with matplotlib (Agg backend), and convert each to base64 PNG.

CLAUDE

Here's the charting section:

```
matplotlib.use("Agg")

def fig_to_base64(fig):
    buf = io.BytesIO()
    fig.savefig(buf, format="png", bbox_inches="tight")
    plt.close(fig)
    return base64.b64encode(buf.getvalue()).decode("ascii")

fig = plt.figure(figsize=(9.5, 3.2))
plt.plot(t, h)
img_header = fig_to_base64(fig)

# ...same pattern for the derivatives, fit-over-data, residuals,
# and area-under-curve figures...
```

Step 7 — HTML dashboard

YOU

Assemble everything into a tabbed HTML dashboard with the embedded charts and stats. JavaScript should only switch tabs - no computation.

CLAUDE

Here's the dashboard assembly:

```
html = f"""<!DOCTYPE html>
<html>...
  
  <table>...R2 = {r2:.4f}...</table>
  <script>
    function showTab(i) {{
      // toggle .active class on the matching tab button and panel
    }}
  </script>
</html>"""

with open("lab02_estiamba.html", "w", encoding="utf-8") as f:
    f.write(html)
```

Step 8 — Console summary

YOU

Print all key results to the console in a format ready to paste into a report.

CLAUDE

Here's the console summary:

```
print(f"max dh/dt = {v_max_rate:.4f} m/h at t={t_max_rate:.2f} h")
print(f"R2={r2:.4f}  SSE={sse:.4f}  s={s_est:.4f}  dof={dof}")
for name, val, s in zip(PARAM_NAMES, popt, se):
    print(f"  {name} = {val:.5f}  SE={s:.5f}")
print(f"quad area={area_quad:.4f} m*h   trapezoid area={area_trap:.4f} m*h")
```